

AI-Powered Response Authenticity Detection for Hiring

Project Requirements and Specifications

Symbol



Symbol Jr.

Damien Flutre

Ziming Fu

Monica Kim

Vsevolod (V) Kovalev

Khang Bui

Table of Contents

I. Introduction	3
II. System Requirements Specification.....	3
II. 1. Use Cases	3
II. 1.1 <i>User Use Cases</i>	5
II. 1.2 <i>Internal Use Cases</i>	6
II. 2. Functional Requirements	8
II.2.1 <i>API Interface and Input Processing</i>	8
II.2.2 <i>Content Analysis and Detection</i>	9
II.2.3 <i>Security and Access Control</i>	10
II.2.4 <i>Dispute Management</i>	10
II.2.5 <i>Testing</i>	10
II. 3. Non-Functional Requirements	10
III. System Evolution	11
IV. Glossary	13

I. Introduction

The increasing adoption of remote and asynchronous hiring has created a new challenge: verifying the authenticity of candidate responses. With powerful generative AI tools widely available, candidates may rely on AI-generated text, read from pre-written scripts, or even play back synthetic speech during interviews. These practices undermine the integrity of the hiring process by obscuring the candidate's true communication skills and in-the-moment reasoning.

This project addresses that gap by developing an AI-powered response authenticity detection service that integrates seamlessly into Symbal's hiring platform. The service will algorithmically assess whether candidate responses are original, scripted, or AI-assisted, across both written and spoken modalities. By fusing evidence from text analysis, audio prosody, and contextual factors, the system will provide hiring managers with a confidence score and supporting indicators. This enables organizations to make more informed and fair hiring decisions while protecting candidates from unfair bias.

The core objective is to deliver a production-ready, scalable API microservice that detects authenticity in both text and audio responses. The system will restore trust in AI-enabled recruiting platforms by giving stakeholders—Symbal, hiring managers, and candidates—transparent and reliable indicators of communication authenticity. Ultimately, this project seeks to balance the efficiency of AI-driven recruitment with the fairness and integrity required in high-stakes decision-making.

II. System Requirements Specification

II. 1. Use Cases

This system supports both **external (user-facing)** and **internal (developer/administrator)** interactions. User-facing use cases include text analysis, audio analysis, and the dispute workflow. Internal use cases include testing the API, accessing the internal FastAPI service, and monitoring usage metrics. Each use case is described in terms of actors, pre- and post-conditions, flows, and related requirements.

Below are two UML use case diagrams displaying both user flows and administrative flows.

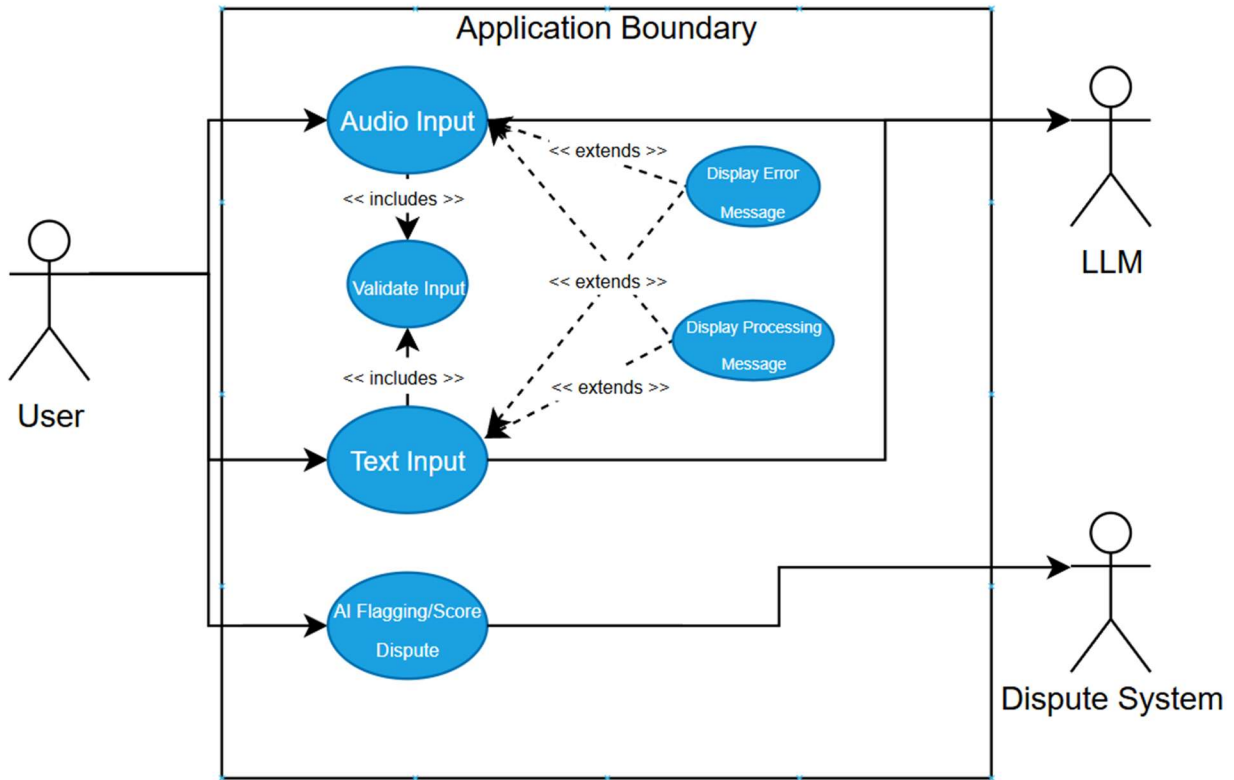


Figure 1: User Use Case Diagram

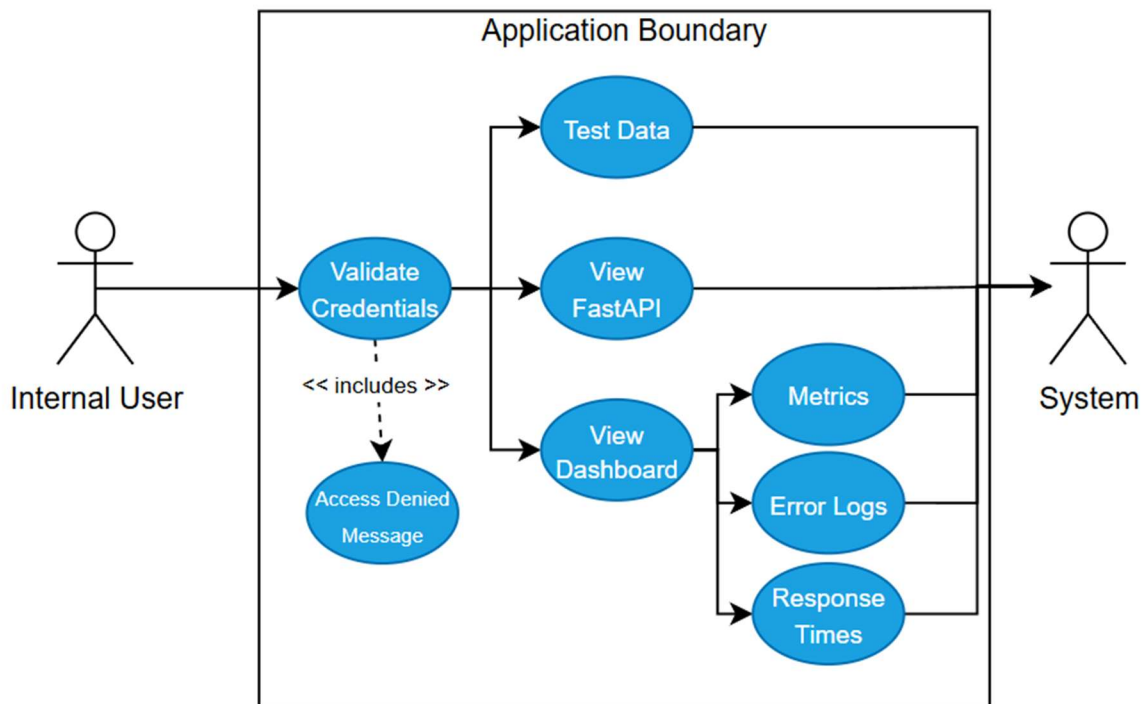


Figure 2: Internal User Use Case Diagram

II.1.1 User Use Cases

Use Case 1: Textual Input Analysis

Use Case	Textual Input Analysis
Actors	Recruiter (primary), Symbal platform (supporting)
Pre-condition	The recruiter has a text response from a candidate interview. The recruiter is authenticated and authorized to use the system.
Post-condition	The system provides a JSON response with a confidence score and label (e.g., <i>Human</i> , <i>AI-generated</i>).
Main Flow	1. The recruiter submits the candidate's text to the <code>/detect/text</code> endpoint. 2. The system validates the input (non-empty, within token limits). 3. The system analyzes the text using linguistic features (perplexity, burstiness, repetition). 4. The API returns a JSON response with a confidence score (0–1) and label. 5. The recruiter views the results.
Alternative Flow	• If the text is empty or exceeds maximum token length, the system rejects the request and returns an error (per FR-04).
Related Requirements	FR-01, FR-02, FR-04, FR-07, FR-04, FR-09

Use Case 2: Audio Input Analysis

Use Case	Audio Input Analysis
Actors	Recruiter (primary), Symbal platform (supporting)
Pre-condition	The recruiter has a recorded audio response in a supported format (.wav, .mp3).
Post-condition	The system provides a JSON response with a confidence score and label (e.g., <i>Human speech</i> , <i>Synthesized</i>).
Main Flow	1. The recruiter uploads the audio file to the <code>/detect/audio</code> endpoint. 2. The system validates the format and quality. 3. The system extracts features such as prosody, disfluencies, and deepfake/TTS signals.

	- The API returns a JSON response with a confidence score and label. - The recruiter views the results.
Alternative Flow	- If the audio file is invalid or unsupported, the system returns an error. - If the file is too large or noisy, the system may reject or return a warning.
Related Requirements	FR-01, FR-03, FR-04, FR-04, FR-09

Use Case 3: Dispute Workflow

Use Case	Dispute Workflow
Actors	Recruiter (primary), Symbol platform (supporting)
Pre-condition	The system has flagged a candidate's response as AI-generated.
Post-condition	A dispute case is created, logged, and recruiter notified.
Main Flow	- The candidate submits a dispute request via /disputes. - The candidate uploads supporting evidence (e.g., original notes). - The system creates a dispute record and marks it as pending. - The recruiter receives a notification (webhook or email). - The recruiter reviews and resolves the dispute.
Alternative Flow	If the candidate has already disputed the result twice, the system rejects the request.
Related Requirements	FR-09, FR-09, NFR-06

II.1.2 Internal Use Cases

Use Case 4: Testing the Internal API

Use Case	Testing the Internal API
Actors	Developer (primary)
Pre-condition	Developer has internal access credentials.
Post-condition	The system returns analysis results, timing statistics, and benchmarks.

Main Flow	1. Developer sends synthetic or user data to the internal testing API. 2. The API processes the data using detection models. 3. The system returns JSON with authenticity results, benchmarks, and runtime statistics.
Alternative Flow	• Invalid or unsupported test data triggers an error response.
Related Requirements	FR-06, NFR-05

Use Case 5: Accessing the Internal FastAPI Service

Use Case	Accessing the Internal FastAPI Service
Actors	Developer (primary)
Pre-condition	Developer has authenticated API key access.
Post-condition	Developer successfully sends and receives data using FastAPI routes.
Main Flow	1. Developer connects to the internal FastAPI environment. 2. Developer invokes test endpoints for debugging or service verification. 3. System responds with processed results.
Alternative Flow	• If the API key is invalid or expired, the system denies access.
Related Requirements	FR-01, FR-06, FR-09

Use Case 6: Monitoring and Logging

Use Case	Monitoring and Logging
Actors	System Administrator (primary)
Pre-condition	Admin is authenticated and authorized.
Post-condition	Monitoring dashboard provides API usage metrics, error logs, and response times.
Main Flow	1. Admin logs into the monitoring dashboard. 2. Admin selects a time range for analysis. 3. Dashboard displays request counts, percentiles, and error rates by endpoint.

	4. Admin downloads error logs in CSV if needed.
Alternative Flow	If authentication fails, the system denies access.
Related Requirements	FR-09, NFR-01, NFR-06

II. 2. Functional Requirements

This section describes the functional requirements which will be implemented to produce the complete system. The functional requirements are sorted according to their function within the system.

II.2.1 API Interface and Input Processing

[FR-01] REST API Endpoints

Description	The system shall provide a REST API with POST endpoints for text (/detect/text) and audio (/detect/audio) input, including required HTTP methods and headers (content-type, API key).
Source	Requirements elicitation from the client.
Priority	Level 0 (Essential)

[FR-02] Text Input Processing

Description	The system shall accept text input (raw string) for authenticity analysis.
Source	Requirements elicitation from the client.
Priority	Level 0 (Essential)

[FR-03] Audio Input Processing

Description	The system shall accept audio input in standard formats (e.g., .wav, .mp3) for authenticity analysis.
Source	Requirements elicitation from the client.
Priority	Level 0 (Essential)

[FR-04] Input Validation

Description	The system shall validate input and reject empty text or text exceeding the max token limit with a clear error message.
Source	Internal requirements elicitation among members of the team.
Priority	Level 0 (Essential)

II.2.2 Content Analysis and Detection**[FR-05] Response Format**

Description	The system shall process text or audio input and return a JSON response with a confidence score between 0-1, and a label (e.g. "AI-generated", "Human", etc.).
Source	Requirements elicitation from the client.
Priority	Level 0 (Essential)

[FR-06] LLM API Usage

Description	The API trims and validates every text payload on text-accepting endpoints. Empty or whitespace-only input is rejected, and input exceeding a tenant-configurable MAX_TOKEN is not processed. Both cases return HTTP 400 with a clear, structured error message indicating the reason and limits.
Source	Team API contract / Reliability requirement
Priority	Level 0 (Essential)

[FR-07] Text Analysis Features

Description	The system shall analyze text input using linguistic features including perplexity, burstiness, and repetition patterns to determine authenticity.
Source	Requirements elicitation from the client.
Priority	Level 0 (Essential)

II.2.3 Security and Access Control

[FR-08] Authentication and Rate Limiting

Description	The system shall require API-key authentication on all endpoints and enforce per-tenant rate limits; requests exceeding limits shall return HTTP 429 with a retry-after header.
Source	Internal requirements elicitation among members of the team.
Priority	Level 0 (Essential)

II.2.4 Dispute Management

[FR-09] Dispute Workflow

Description	The system shall provide a dispute and review workflow via REST endpoints (/disputes) to create a dispute, upload supporting evidence, query status, and notify recruiters via webhook or email on status changes.
Source	Internal requirements elicitation among members of the team.
Priority	Level 1 (Desirable)

II.2.5 Testing

[FR-10] Testing Framework

Description	The system shall have a separate testing API or framework configured for internal development use.
Source	Internal requirements elicitation among members of the team.
Priority	Level 1 (Desirable)

II. 3. Non-Functional Requirements

This section describes the non-functional requirements which define the system properties and constraints. These non-functional requirements will guide the implementation of the functional requirements.

Non-Functional Requirement	Description
[NFR-01] Scalability	The system shall be deployable as a microservice, capable of handling multiple simultaneous requests without performance degradation.
[NFR-02] Containerization	The system shall be containerized (e.g., Docker) to support deployment across cloud and on-premise environments.
[NFR-03] Cost Efficiency	The system shall use the most cost-efficient LLM API to minimize costs per API call.
[NFR-04] Cost Control	If using an outside LLM API, there shall be a cost ceiling configured to avoid incurring large processing costs.
[NFR-04] Performance	The system shall process and return results for text input within 3 seconds and audio input within 10 seconds under normal load conditions.
[NFR-05] Maintainability	The system shall be built with separate text detection, audio detection, and API service layers to enable independent maintainability.
[NFR-06] Security	All data in transit shall be encrypted with TLS 1.3, and all stored text, audio, and log artifacts shall be encrypted at rest with AES-256.

III. System Evolution

A. Baseline Assumptions

- Access to interview text/audio for training and evaluation; English first.
- Containerized deployment with GPU capacity within cost limits (NFR-02, NFR-03, NFR-04).
- Third-party TTS/voice models permit internal dataset generation.
- Authentication, rate limiting and monitoring (FR-09, FR-10, NFR-06).

B. Planned Evolution

Phase 0: Data Strategy

- Evaluate existing voice/TTS models for controllable synthesis of mumbling, accent, slow/fast rate, and scripted-reading simulation.
- If quality is sufficient, generate N labeled samples with ground-truth factors to seed training.
- If not, use pre-recorded human reads plus audio augmentation (speed, noise, codec, reverb).

Phase 1: Dual-Path Audio Detector

- Path A (Disfluency/Naturalness): fine-tuned lightweight RNN/Conformer to capture “um/uh,” pauses, and prosody variability; output a normalized score and features.
- Path B (Script/Reading Likelihood): fine-tuned RNN or modern encoder to detect alignment patterns typical of reading or playback
- Fusion: combine both scores into a single confidence (0-1) and label via a simple rule or a small calibrator model.

Phase 2: Text + Audio Fusion

- Add text features (perplexity, burstiness, repetition; FR-07) and fuse with audio scores. Optionally include an evidence block with per-path contributions (backward compatible).

Phase 3: Scale and Languages

- Add language/accent packs as data becomes available.
- Reduce latency and cost via quantization (NFR-03, NFR-04, NFR-05).

C. Change Drivers

- Rapid shifts in TTS/deepfake and speech encoders.
- Variation by accent, microphone, and speaking rate across clients; manage data drift.
- Policy or consent updates affecting processing.
- Tighter SLAs or budgets requiring smaller/faster models.

D. Data, Metrics, and Retraining

Datasets: synthetic TTS set with labeled controls; augmented human set; hard negatives from disputes and misclassifications.

Metrics: accuracy/AUC, false-positive rate at high recall, calibration error, and cost per request.

Monitoring: track score distributions; alert on drift

E. Risks and Mitigations

- Insufficient TTS control → use pre-record + augmentation
- Overfitting to synthetic artifacts → mix real data
- Latency/cost overruns → quantize
- Regulatory constraints → minimize retention
- Bias across accents/languages → targeted data collection

F. Stable Contracts

- Public REST endpoints and JSON response remain stable (FR-01, FR-04).
- Authentication, rate limiting, and logging remain unchanged (FR-09, NFR-01, NFR-06).
- Versioning and deprecation: model/API versions with a defined migration window remain unchanged (FR-06).

IV. Glossary

API (Application Programming Interface)

A set of defined rules that allows different software systems to communicate. In this project, the API enables recruiters and developers to submit candidate responses for authenticity analysis.

REST (Representational State Transfer)

A widely used web service style that relies on standard HTTP methods (e.g., POST). The detection system provides REST endpoints for text and audio input.

Endpoint

A specific URL exposed by the API where a function can be accessed. Examples include /detect/text for textual analysis and /detect/audio for audio analysis.

JSON (JavaScript Object Notation)

A lightweight, text-based format used for structured data exchange. The system returns results (confidence scores and labels) in JSON format.

Confidence Score

A numeric value (0–1) that indicates the likelihood a response is human, AI-generated, or synthetic speech. Higher values represent stronger confidence in the classification.

Label

The categorical output assigned to a response after analysis, such as “Human,” “AI-generated,” or “Synthesized.”

Token

A basic unit of text (e.g., a word or sub-word). Token limits are enforced to restrict the maximum input size sent to the API.

Perplexity

A linguistic measure of how predictable a sequence of text is for a language model. Low perplexity may suggest machine-generated text, while higher values indicate more natural human writing.

Prosody

The rhythm, pitch, and intonation patterns in spoken language. Human prosody typically includes natural variations, while synthetic speech often lacks them.

Disfluencies

Natural speech interruptions such as “um,” “uh,” or pauses. Their presence can be an indicator of genuine human speech versus scripted or AI-generated audio.